Master 2 Electronique Energie Electrique Automatique.

Parcours : Système et Microsystèmes Embarqué

Année 2024-2025

De l'Université de TOULOUSE III PAUL SABATIER

Rapport de bureau d'étude

Synthèse et mise en œuvre des systèmes

Intitulé : Pilote de barre franche

Rédigé par :

LELLOUCHI Mehdi

BOUTAHAR Mohammed

Encadré par :

M.Thierry Perissé

Table des matières

Introduction : 3
Description du Système 3
Environnement du travail : logiciels/materiel : 4
Présentation Quartus : 4
Présentation ModelSim : 5
Carte DE0 nano : 5
Platform designer 5
nios 2: 6
Fonction simple anémomètre : 6
Définition de l'anémomètre : 6
Principe de fonctionnement de l'anémomètre 7
Schéma fonctionnel : 8
Liaison fichiers en vhdl quartus : ..... **Erreur ! Signet non défini.**
Simulation modelsim : 9
Validation des résultats sur l'oscilloscope : 9
Sopc : 14

Introduction :

L'automatisation de la navigation maritime constitue un enjeu majeur pour améliorer les performances et la sécurité des systèmes embarqués. Dans le cadre de ce bureau d'étude et dans l'unité d'enseignement intitulé « synthèse et mise en œuvre des systèmes » nous avons conçu et implémenté un système de pilotage automatique pour voilier, centré sur la gestion de la barre franche via un vérin et des capteurs essentiels. L'objectif global est de permettre au voilier de naviguer de façon autonome grâce à un FPGA en intégrant des capteurs, des commandes et une interface de contrôle.

Pour atteindre cet objectif, nous avons suivi une méthodologie rigoureuse, structurée en plusieurs étapes :

Analyse des besoins : on a travaillé sur deux fonctions :

Une simple : mesure de vitesse du vent et la gestion de l'interface utilisateur a l'aide des boutons et des leds. Pour but de faire à la fin un asservissement de toutes les fonctions du système.

Découpage fonctionnel on a identifié les modules nécessaires pour réaliser nos fonctions, ainsi que les différents flux de données (signaux et interconnections).

Création et description des **machines à états** pour assurer le fonctionnement séquentiel et la logique de commande du système notamment sur la partie de la gestion de boutons et Leds.

Implémentation en VHDL pour implémenter les différentes fonctionnalités telles que la gestion de l'anémomètre, des **boutons et LEDs**, et le pilotage du vérin.

Utilisation de **Platform Designer** pour intégrer des composants et faciliter la communication entre les différents modules du projet.

Codage en C sur NIOS II pour le traitement des données et la gestion des interfaces au niveau logiciel.

Intégration finale des composants dans un système SOPC (**System on Programmable Chip**) pour garantir la communication et le fonctionnement global du système.

Ce rapport présente les différentes étapes de réalisation du projet, depuis l'analyse des besoins jusqu'à l'intégration sur le FPGA, en détaillant les aspects fonctionnels et les résultats obtenus.

Description du Système

Le système de pilotage de barre franche est un dispositif automatisé qui utilise plusieurs capteurs et composants pour assurer le contrôle de la direction. Il intègre une **interface homme-machine (IHM)** composée de boutons et de LEDs permettant l'interaction avec l'utilisateur. Le cap est mesuré à l'aide d'un **capteur CMPS03** via une interface **PWM** ou

I2C, tandis que la **vitesse du vent** est fournie par un **anémomètre** en mesurant la fréquence. Le système reçoit également des **trames NMEA** provenant d'un **capteur GPS** pour le cap et la position. Toutes ces données sont traitées par le **système de pilotage**, qui envoie des **commandes PWM** au **vérin linéaire** pour ajuster l'angle de la barre franche, avec un retour de **position vérin** pour assurer un contrôle précis. Cette architecture permet une régulation automatique et efficace en fonction des informations de cap et des conditions environnementales, tout en facilitant l'interaction utilisateur.

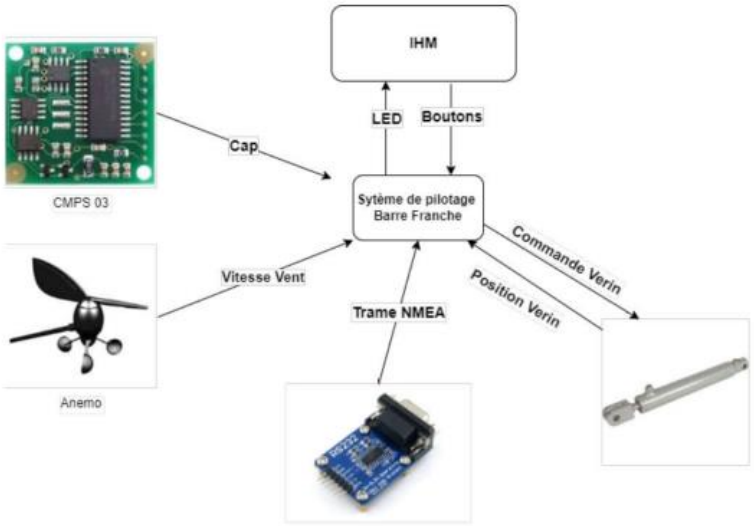


Figure 1 : système de pilotage de barre

Environnement du travail : logiciels/materiel :

Présentation Quartus :

Quartus est un logiciel développé par la société Altera, permettant la gestion complète d'un flot de conception CPLD ou FPGA. Ce logiciel permet de faire une saisie graphique ou une description HDL (VHDL ou verilog) d'architecture numérique, d'en réaliser une simulation, une synthèse et une implémentation sur cible reprogrammable.

Il comprend une suite de fonctions de conception au niveau système, permettant d'accéder à la large bibliothèque d'IP d'Altera et un moteur de placement-routage intégrant la technologie d'optimisation de la synthèse physique et des solutions de vérification. De manière générale, un flot de conception ayant pour but la configuration de composants programmables se déroulent de la manière suivante

https://www.polytech.umontpellier.fr/pravo/cours/Logique/Altera/Notice%20Quartus.pdf

Présentation ModelSim :

ModelSim, développé par Mentor Graphics, est un simulateur HDL (Hardware Description Language) largement utilisé pour la simulation et la vérification des circuits numériques dans les ASIC et FPGA. Il prend en charge plusieurs langages, notamment Verilog, SystemVerilog, VHDL et SystemC, et permet des simulations mixtes entre ces langages. Doté d'une interface graphique avancée, il propose des outils de débogage, un visualiseur de formes d'ondes et une exploration hiérarchique des designs. Il s'intègre parfaitement avec des environnements tels que Quartus II, Vivado, ISE, et même MATLAB, ce qui en fait un choix privilégié pour les développeurs travaillant sur des systèmes complexes.

https://www.researchgate.net/profile/Hamza-Atoui/publication/341406706_TESTBENCH_bancs_de_test_et_le_logiciel_ModelSim_pour_les_masters_II_SEM/links/5ebe90d7299bf1c09abc5696/TESTBENCH-bancs-de-test et-le-logiciel-ModelSim-pour-les-masters-II-SEM.pdf

Carte DE0 nano :

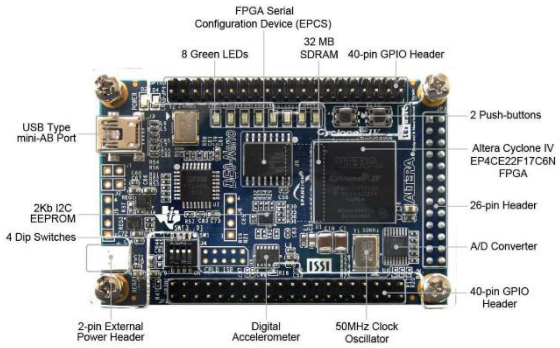


Figure 2 : Carte DE0 nano

La carte de développement **DE0-Nano** est une plateforme FPGA compacte et polyvalente, idéale pour le prototypage de circuits dans des projets tels que les robots ou les dispositifs portables. Elle offre des interfaces d'extension grâce à deux GPIO externes et intègre une mémoire SDRAM et EEPROM, ainsi que des périphériques comme des LED et des boutons-poussoirs pour faciliter la conception. Légère, reconfigurable et sans besoin de matériel additionnel, la DE0-Nano convient particulièrement aux applications mobiles où la gestion de la puissance est cruciale, grâce à ses trois options d'alimentation : USB mini-AB, broches pour alimentation externe et broches 5V DC.

Plateforme designer **:**

Le Platform Designer est l'outil d'intégration système de nouvelle génération dans le

logiciel Intel® Quartus® Prime. Le Platform Designer permet d'économiser du temps et des efforts considérables dans le processus de conception du FPGA en générant automatiquement une logique d'interconnexion pour connecter les fonctions et sous-

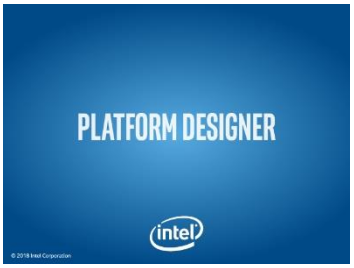


Figure 3 : Plateforme designer

systèmes de propriété intellectuelle (IP).[https://www.intel.fr/content/www/fr/fr/software/programmable/quartus-prime/qts-platform-designer.html]

Platform Designer a été utilisé pour concevoir notre SoPC en intégrant les composants nécessaires et en définissant leur interconnexion. Nous avons créé des blocs matériels personnalisés, assigné les adresses de base pour assurer une communication efficace entre les éléments, et développé des composants spécifiques répondant aux besoins de notre projet. Cet outil a permis une conception structurée et modulable, facilitant l'intégration et la personnalisation du système.

nios 2:

Le **Nios II** est un processeur softcore 32 bits de type RISC développé par Altera (aujourd'hui Intel) pour une implémentation sur leurs FPGA. En tant que processeur softcore, il est défini en langage de description matériel, ce qui permet une personnalisation selon les besoins spécifiques de l'application. Cette flexibilité le rend adapté à une variété d'applications, allant du traitement numérique du signal aux systèmes de contrôle.

Pour le développement avec le Nios II, Intel propose le **Nios II Embedded Design Suite (EDS)**, un ensemble d'outils inclus dans le logiciel Quartus. Le Nios II EDS offre un environnement complet pour la conception matérielle et logicielle, permettant aux développeurs de créer, tester et déployer des systèmes embarqués basés sur le processeur Nios II. Il comprend des outils pour la configuration du processeur, la génération de systèmes, la compilation du code, ainsi que des fonctionnalités de débogage et de simulation. Cette suite facilite l'intégration du processeur Nios II dans des conceptions FPGA, optimisant ainsi le flux de développement pour les applications embarquées.[Guide de démarrage du kit de développement Nios II - Intel]

Fonction simple anémomètre :

Définition de l'anémomètre :

Un anémomètre est un dispositif utilisé pour mesurer la vitesse du vent, indispensable dans divers domaines tels que la météorologie, l'aviation, et les systèmes embarqués maritimes ou terrestres. Dans le cadre du projet, l'anémomètre sélectionné joue un rôle clé en fournissant des informations précises sur la vitesse du vent pour le système embarqué du pilote de barre franche. Ce capteur génère une sortie sous forme de fréquence logique variable, proportionnelle à la vitesse du vent, et est conçu pour une intégration facile dans les systèmes électroniques.



Figure 4 : Anémomètre

https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.meteo-shopping.com%2Ffr%2Fcapteur-a-raccorder%2F237-anemometre-int10ba-0-10v.html&psig=AOvVaw0LuZ9X3rpGkeDnjcmDd1Uz&ust=1734386161560000&source=images&cd=vfe&opi=89978449&ved=0CBQQjRxqFwoTCJCxgoziqooDFQAAAAAdAAAAABAE

Principe de fonctionnement de l'anémomètre

L'anémomètre fonctionne en convertissant l'énergie mécanique du vent en un signal électrique exploitable. Lorsque le vent fait tourner les palettes ou le rotor relié à un alternateur, un signal électrique proportionnel à la vitesse du vent est généré. Ce signal est ensuite converti en une fréquence logique, variant de 0 à 250 Hz, correspondant à une plage de vitesse de 0 à 250 km/h. Ce dispositif ne nécessite pas d'alimentation externe et offre une grande robustesse grâce à sa conception mécanique et ses protections électriques conformes aux normes EMI (EN50081 et EN50082). En outre, il est protégé contre les conditions environnementales difficiles avec une certification IP65, garantissant un fonctionnement fiable dans une plage de température allant de -30 °C à +70 °C. Ce capteur est ainsi parfaitement adapté à une utilisation dans un

système embarqué, où sa sortie numérique facilite le traitement des données par des interfaces électroniques.

Schéma fonctionnel :

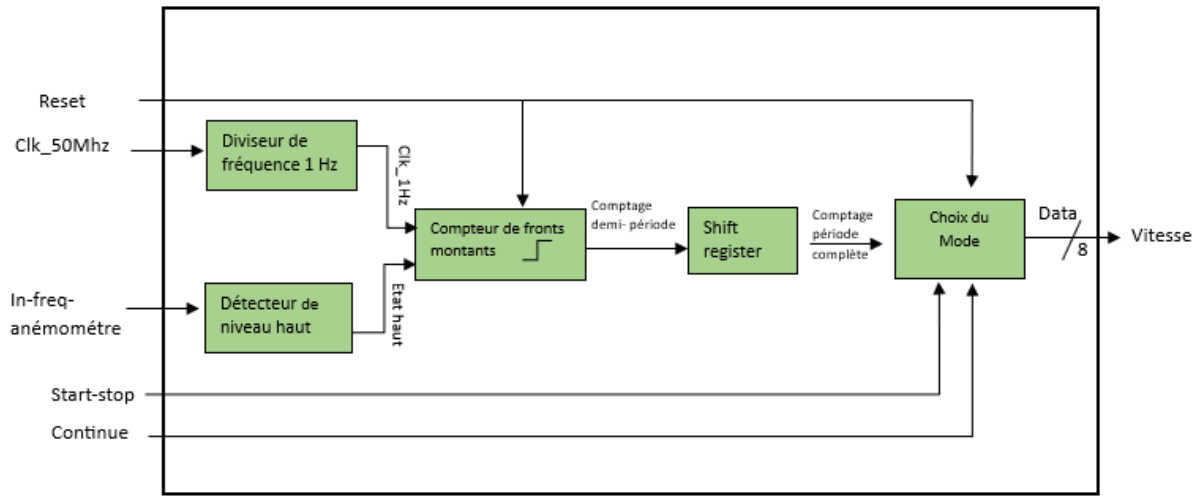


Figure 5 : Schéma fonctionnel

Notre anémomètre aura 3 entrées : l'horloge de 50 Mhz, le reset pour la réinitialisation des différentes parties et la fréquence d'entrée de l'anémomètre qu'on convertira par la suite à sa valeur équivalente en vitesse.

Nous présentons par la suite le rôle des différents blocs de notre schéma fonctionnel :

Diviseur de fréquence : ce bloc réduit la fréquence d'horloge principale (50 MHz) pour générer une horloge plus lente d'une fréquence de (1 Hz). On utilisera Cette horloge pour cadencer le comptage des fronts montants du signal d'entrée de la fréquence de l'anémomètre, pour pouvoir mesurer sa période avec précision.

Détecteur de niveau haut : ce bloc surveille le signal in-freq-anémomètre provenant de l'anémomètre et génère un signal logique lorsqu'il est à un **niveau haut**. Ce signal lancera par la suite **le compteur de fronts montants**, ce qui va nous permettre de mesurer la durée pendant laquelle le signal reste haut sur une période. Nous exploitons cette mesure pour avoir la fréquence de ce signal.

Compteur de fronts montants : ce bloc fait le comptage du nombre de **fronts montants** détectées sur le signal d'entrée de l'anémomètre en état haut pendant une

période définie par l'horloge **1 Hz**. Cette valeur représente la **demi-période** du signal de fréquence.

Registre de décalage : ce bloc assure la multiplication par 2 la valeur du **compteur de fronts montants** en effectuant un **décalage à gauche** d'un bit. Avec cette opération on aura la valeur de la **période complète** du signal de l'anémomètre, puisque la valeur du compteur correspond à une **demi-période**.

Choix du mode : ce bloc reçoit la valeur de la **période complète** calculée (en sortie du registre de décalage) et la convertit en une valeur exploitable, correspondant à la **vitesse du vent**.

Simulation modelsim :

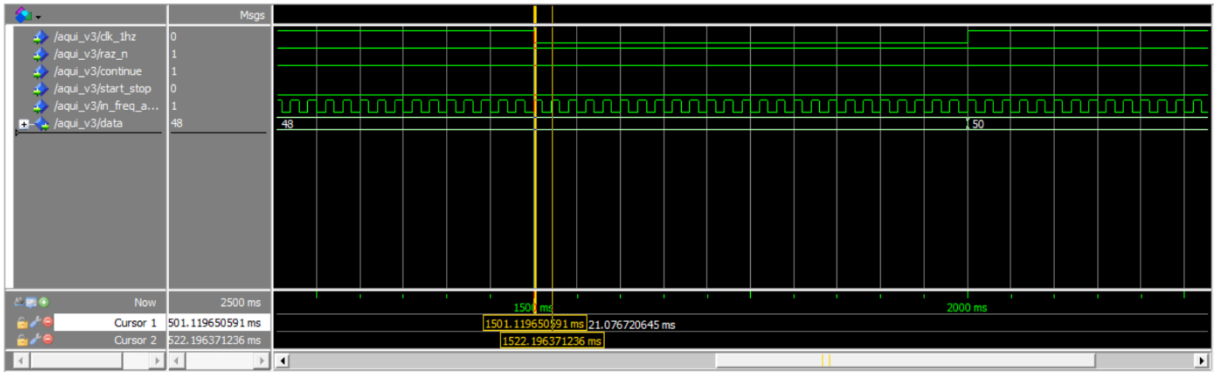


Figure 6 : simulation d'anémomètre sur modelsim

Validation des résultats sur l'oscilloscope :

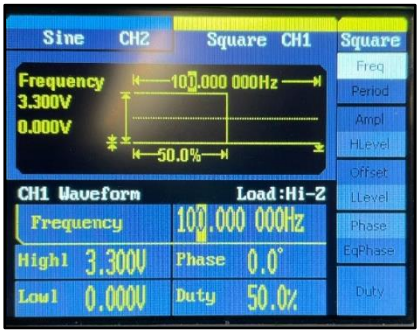


Figure 7 : consigne de 100 Hz

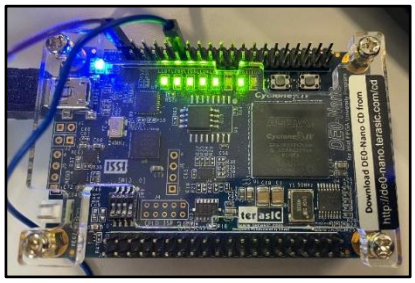


Figure 8 : résultat affiché en codage 8 bits sur des LEDS

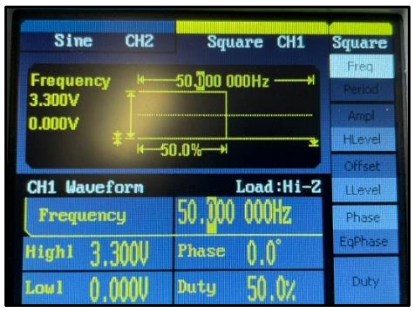


Figure 9 : consigne de 50 hz

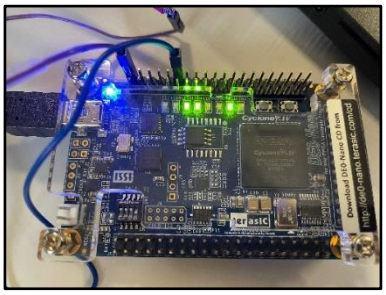


Figure 10 : résultat affiché en codage 8 bits sur des LEDS

Fonction complexe : gestions des boutons et LEDs :

Définition de la fonction :

Dans cette section, nous allons intégrer des boutons-poussoirs pour permettre un contrôle efficace de notre pilote. Trois boutons seront utilisés : Bâbord, Tribord et Stby/Auto, dont les fonctions spécifiques seront détaillées ultérieurement.

Pour compléter cette interface, des LEDs et un buzzer seront ajoutés, offrant des retours visuels et sonores afin d'améliorer l'interaction avec le système.

Le pilote à barre franche propose plusieurs modes de fonctionnement, activés via les boutons-poussoirs. Chaque combinaison génère un code de sortie, appelé "code_fonction", codé sur 4 bits et correspondant à l'un des 7 modes de fonctionnement disponibles.

Cette approche permet de gérer facilement et efficacement les différentes configurations du pilote.

Pour implémenter cette fonction, nous la décomposons en plusieurs sous-fonctions, chacune correspondant à un composant spécifique. Ces composants incluent :

Un composant pour gérer l'allumage des LED,

Un composant pour générer le bip sonore,

Un composant pour gérer la temporisation,

Un composant pour traiter les boutons d'entrée.

Ces modules seront intégrés au sein d'un composant global, chargé de coordonner l'ensemble pour réaliser la fonctionnalité souhaitée.

Ce signal est généré par la machine à états du composant de gestion des boutons. En sortie, le composant fournit un signal pilotant directement les LED.

Génération de temporisation

Pour distinguer un appui long d'un appui court sur un bouton, un composant de temporisation est utilisé. Lorsqu'il est activé via le signal a_tempo, ce composant génère après 3 secondes un signal t_3s. Ce signal permet d'interpréter les actions suivantes :

Si le bouton est maintenu enfoncé et que t_3s est également à 1, il s'agit d'un appui long.

Si le bouton est relâché avant les 3 secondes, il s'agit d'un appui court.

Génération du bip sonore

Pour produire un signal sonore, un composant dédié est conçu. Ce composant prend un signal sur 2 bit "bip_mode" en entrée et génère en sortie :

Un signal bip_out de durée 250 ms, utilisé pour produire le bip sonore simple.

Un signal bip_out de durée 250ms avec un silence de 100 ms suivie d'un signal de 250 ms pour un bip double.

Gestion des boutons

Le composant de gestion des boutons traite les entrées provenant de trois boutons distincts. Les entrées incluent :

- Trois signaux correspondant aux états des boutons.
- Un signal de fin de temporisation, permettant de déterminer si l'appui est court ou long.

En sortie, le composant génère :

Trois signaux LED codés sur 2 bits, utilisés par le module de gestion des LED pour définir leurs états.

Un signal bip_out pour activer la génération du bip.

Un signal a_tempo pour activer la temporisation.

Un signal code_Fonction codé sur 4 bits, représentant la fonction sélectionnée par la commande.

Schéma fonctionnel :

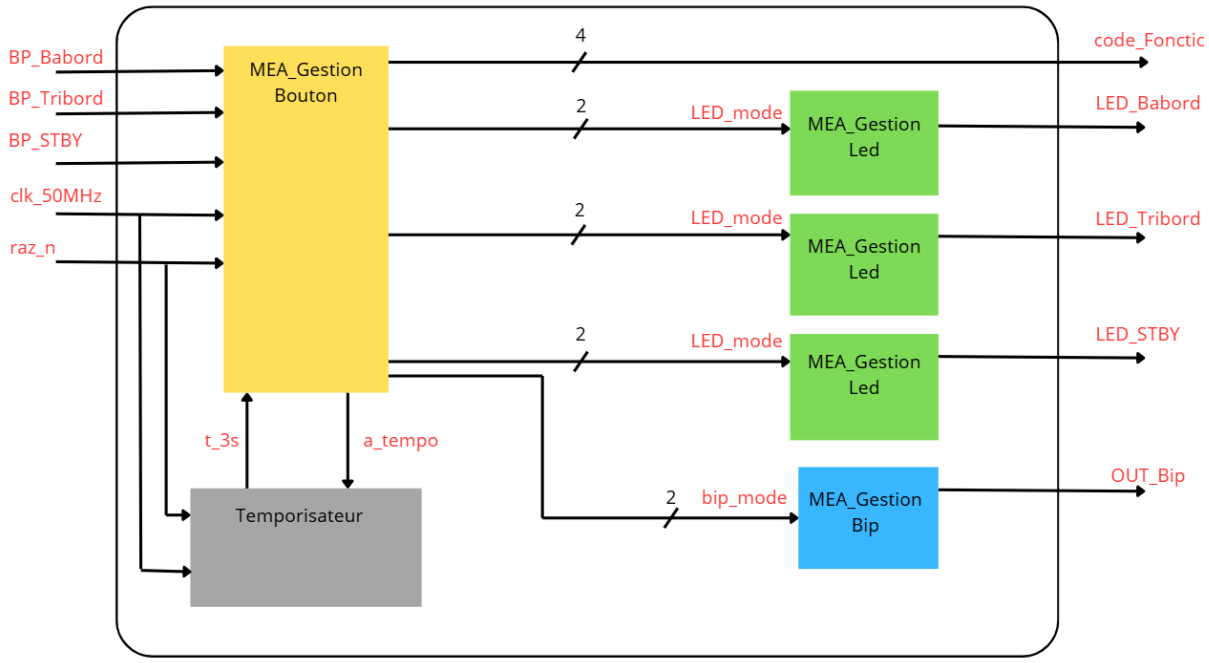


Figure 11 : schéma fonctionnel gestion boutons et LEDS

Machine a état :

On a fait on sorte de separer les machines a etat afin de reduire la complexité de cette dernière. Est donc on a 3 machines a etat : une machine a etat complete qui integre la gestion des boutons, la commande des Leds et Bips, le bascule entre le mode manuel et le mode automatique, et aussi qui donne en sortie un code fonction qui servira de consigne pour une autre fonction par la suite :

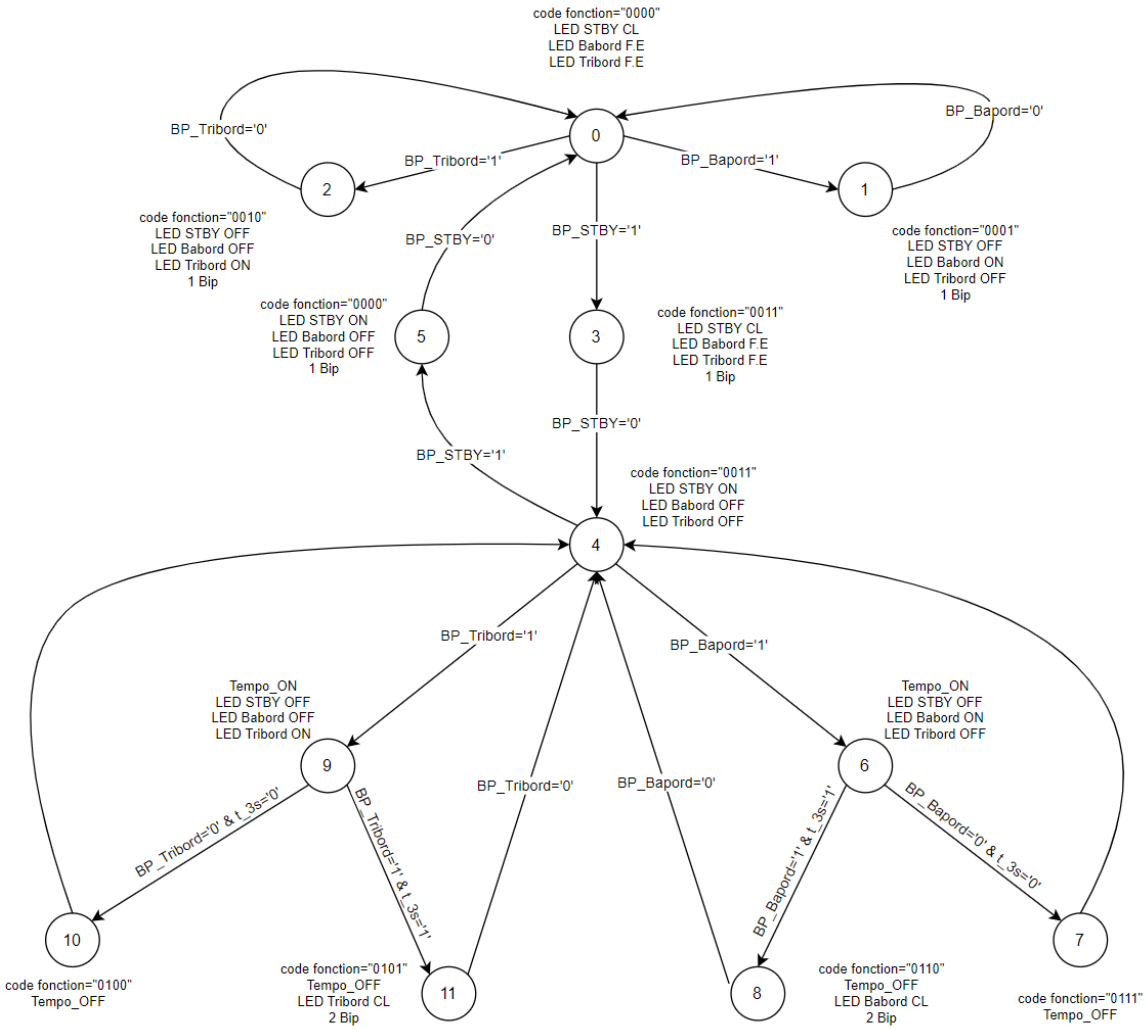


Figure 12 : machine a état gestion boutons

Simulation modelsim machine a état gestions boutons :

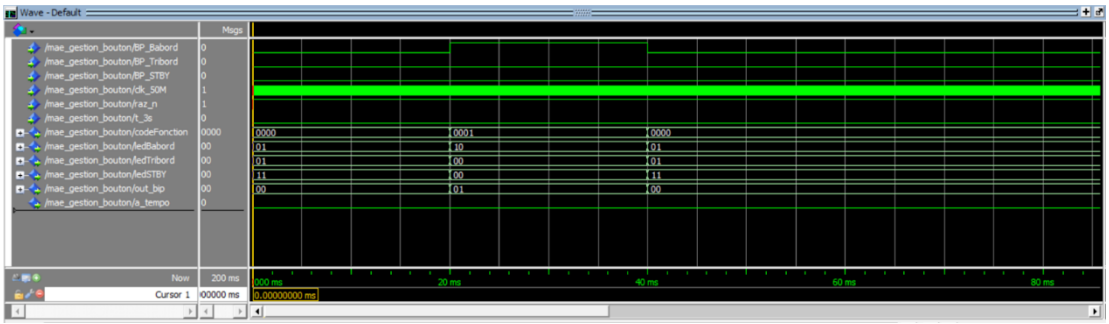


Figure 13 : simulation modelsim

Machine a état gestions des Leds :

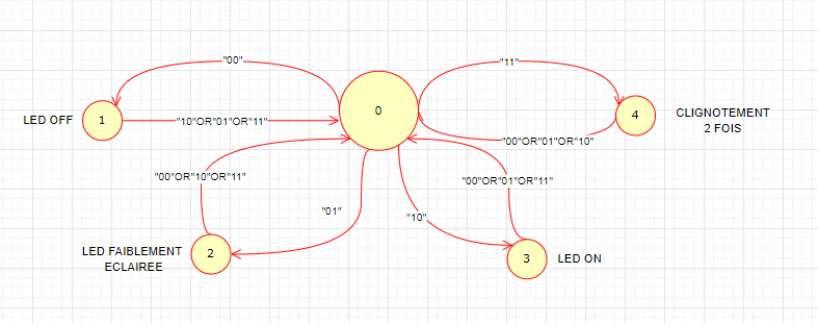


Figure 14 : Machine a état gestions des Leds

Simulation de la machine a état gestions des Leds :

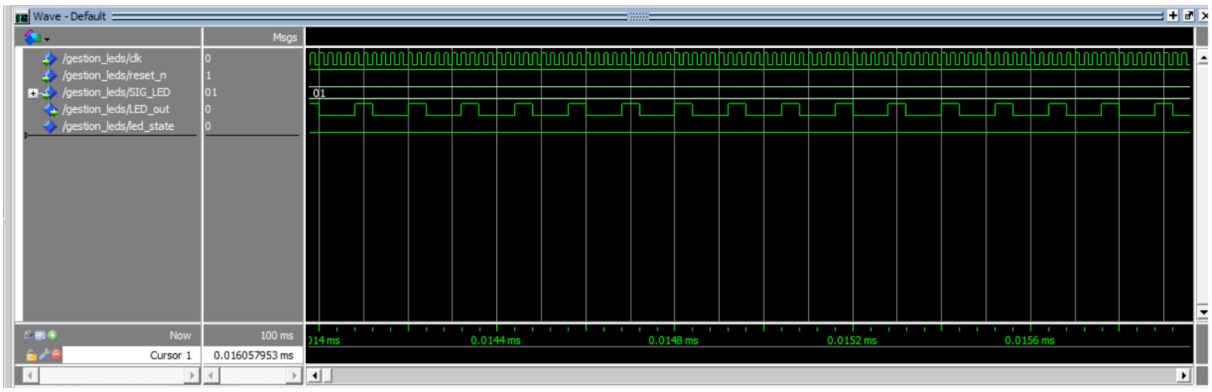


Figure 15 : simulation modelsim gestion boutons

Machine a état gestions des bips :

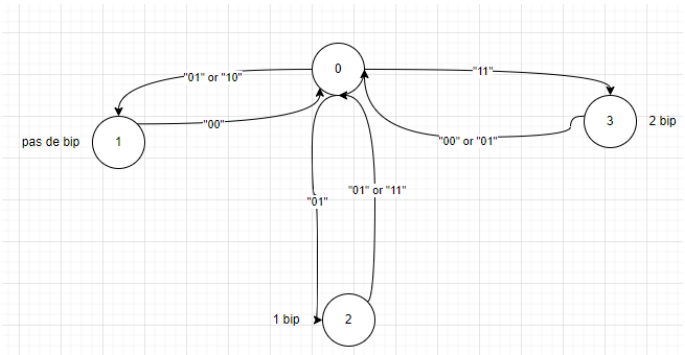


Figure 16 : machine a état gestion des bips

Simulation Machine a état gestions des bips :

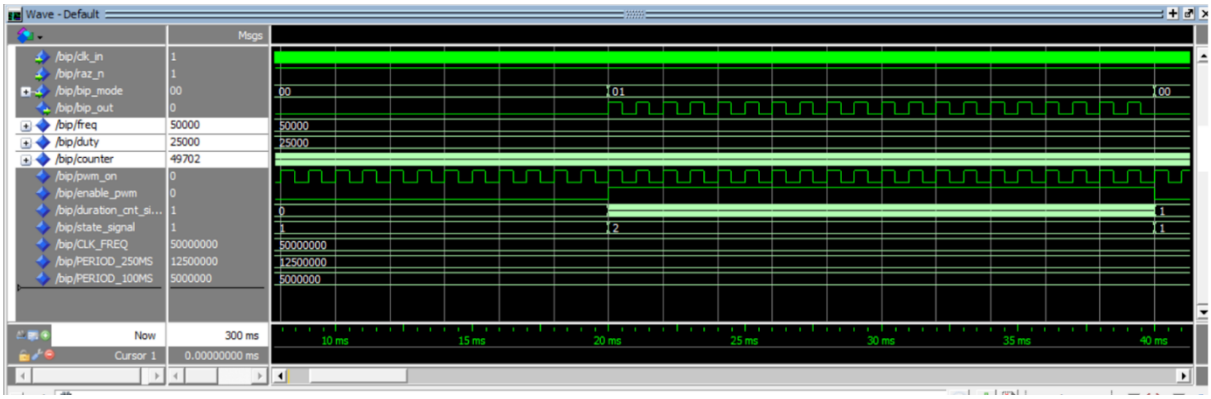


Figure 17 : simulation modelsim gestion bips

Validation des résultats de la fonction complexe

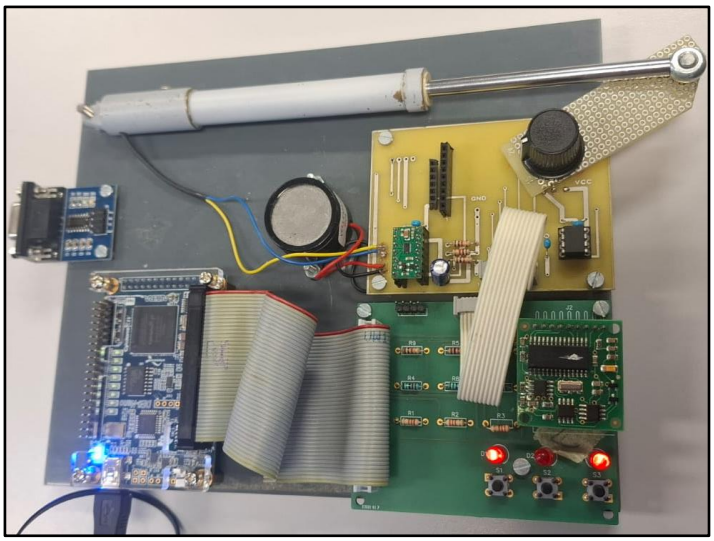


Figure 18 : résultat sur la carte

A cette étape on a pu faire la vérification expérimentale de toutes les chemins possibles de notre machine a état.
Toutes les fonctionnalités fonctionnent parfaitement comme prévu, l'allumage des LEDS avec les états présentés sur leur machine a état, le son du bip

On a pu aussi faire fonctionner notre vérin avec le code fourni par le professeur, on a pu adapter la liaison entre les fonctions, en récupérant le code fonction et en l'exploitant pour commander le vérin.

Sopc :

Le **SOPC** (System On Programmable Chip) est une architecture matérielle et logicielle intégrée, basée sur un FPGA, permettant de concevoir des systèmes embarqués complexes en combinant des composants matériels spécialisés et un processeur logiciel. Dans ce projet, le SOPC repose sur une carte DE0 Nano équipée d'un FPGA Cyclone IV, servant de plateforme de développement. Il permet d'intégrer plusieurs interfaces et modules nécessaires au fonctionnement du pilote de barre franche.

Voici les principaux éléments et leur intégration dans le SOPC :

Interfaces matérielles :

Anémomètre : Acquisition de la vitesse du vent sous forme d'une fréquence logique, traitée pour fournir une donnée exploitable.

Commande de vérin : Gestion de la position de la barre via un signal PWM, intégrant la détection des butées de fin de course.

Interface utilisateur : Gestion des boutons poussoirs, LEDs et du buzzer pour l'interaction avec le barreur.

Traitement matériel et logiciel :

Le SOPC permet un **partitionnement matériel/logiciel** optimisé. Les tâches critiques en temps réel, comme la gestion des signaux PWM ou les conversions des capteurs, sont prises en charge par des modules matériels spécialisés. Les fonctions moins critiques, telles que la gestion des états du système et l'interprétation des commandes utilisateur, sont gérées par un processeur logiciel (NIOS II).La communication entre les modules est assurée par un **bus Avalon**, garantissant une interconnexion efficace des composants matériels et des périphériques.

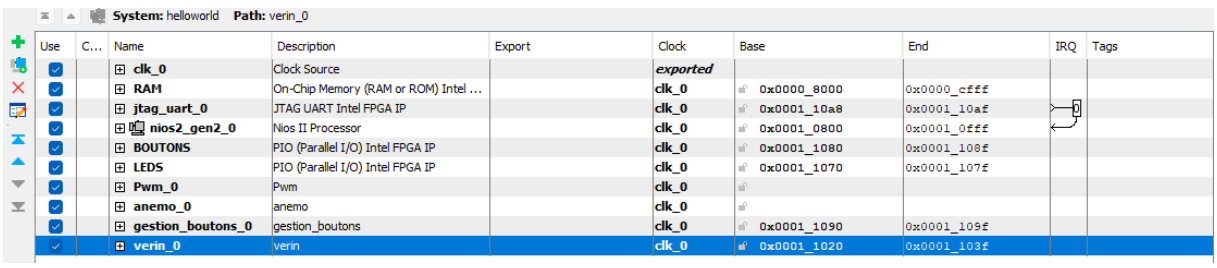


Figure 19 : composants du sopc sur platform designer

Ce SOPC (System-on-Programmable-Chip) a été créé à l'aide de **Platform Designer** pour intégrer différents composants matériels interconnectés.

Horloge principale (clk_0) :

Fournit une horloge à l'ensemble du système.

Mémoire RAM :

Utilisée pour stocker des données ou des instructions.

Connectée au bus Avalon.

Processeur Nios II (nios2_gen2_0) :

Exécute le programme et contrôle les périphériques.

Interface JTAG UART :

Permet la communication avec le système pour le **débogage**.

Connexions Avalon : Les composants sont reliés via le bus **Avalon**, facilitant la communication entre le processeur, la RAM et les périphériques.

Validation des résultats après l'intégration des fonctions sur le sopc :

Fonction simple sur sopc :

La donnée a laquelle nous intéresserons à l'issue de cette fonction est la vitesse, on affichera sur la console d'Eclipse Nios II cette vitesse relative a un signal carre de 100 Hz .

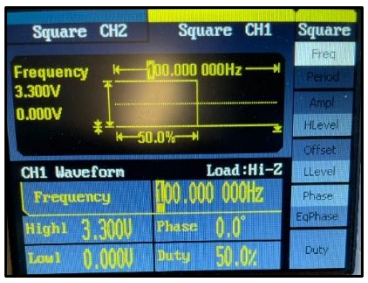


Figure 20 :consigne de 100 HZ

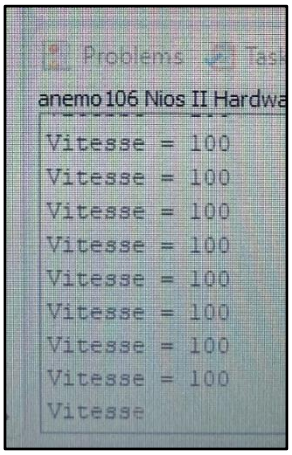


Figure 21 :résultat sur le console

On constate qu'on a bien la valeur exacte sur notre console.

Fonction complexe sur sopc :

Apres l'intégration de la fonction de gestion des boutons et des Leds dans notre sopc, on regardera le fonctionnement qui est parfaitement adéquat a ce qui est décrit dans le cahier de charge et ce qu'on a essayer de représenter avec nos machines a état.

On affichera sur notre console les codes fonction aux sorties de chaque état qui seront lues par la suite par une autre fonction et qui serviront a commander le vérin.

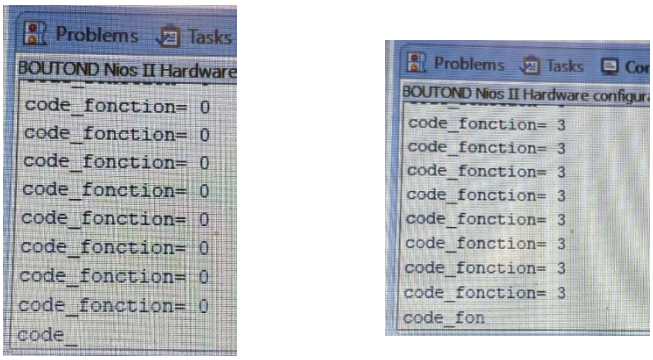


Figure 22 : code fonction affiché sur la console

Conclusion :

Ce projet de bureau d'étude a constitué une expérience particulièrement enrichissante, nous permettant de développer et consolider nos compétences en électronique numérique et en programmation VHDL. Nous avons pu analyser les spécifications, découper le système en blocs fonctionnels, concevoir des circuits d'interfaces numériques, et valider leur fonctionnement par simulation et sur maquette à l'aide d'outils performants tels que la carte DE0 d'Altera.

Le projet nous a également permis de travailler sur des notions avancées comme le co-design, l'interface avec un bus microprocesseur (NIOS et bus Avalon), et la conception d'un SOPC intégrant des IP propriétaires et tiers. La validation des différentes parties, que ce soit en simulation ou via des tests pratiques, nous a confrontés aux défis réels de la conception et de l'implémentation des systèmes électroniques complexes.

Grâce à cette approche, nous avons non seulement acquis des compétences techniques précises, mais également développé une compréhension approfondie des processus d'implémentation matérielle et des limites imposées par le hardware.